BACKEND CONCEPTS GUIDE

Brief! - the Java and Spring technologies behind it

For Brief! 2.2.0 (Spring Boot edition) | Study companion to the code

What this is	Each technology used in the Brief! backend, explained once: what it is, what problem it solves, and the file in Brief! where you used it. Written to be read next to the study copy.
How to use it	Read one section. Open the files it names. Then say the green box aloud without looking. If you can, you can defend that part of the project in an interview.
Covers	Java 17, Maven, Spring Framework (IoC, beans, dependency injection), Spring Boot, Spring MVC, Bean Validation, JDBC, JPA, Hibernate, Spring Data JPA, transactions, Spring Security, JWT, PBKDF2 password hashing, rate limiting, JUnit 5, Mockito.
Not covered	The web app (TypeScript, React, Vite). That is a separate guide.
Versions	Taken from your own test run of October 6, 2026: Java 17.0.11, Spring Boot 3.3.5, Spring Security 6.3.4, Hibernate 6.5.3, Tomcat 10.1.31.

#	Section
1	The Whole Stack on One Page
2	Spring Framework: IoC, Beans, Dependency Injection
3	Spring Boot
4	The Web Layer: Spring MVC, JSON, Validation
5	Persistence: JDBC, JPA, Hibernate, Spring Data JPA
6	Transactions, Indexes, Constraints
7	Security: Passwords, Spring Security, JWT, Rate Limiting
8	Testing: JUnit 5, Mockito, Spring Boot Test
9	Maven and Java 17
10	Interview Answers in Two Sentences

© 2026 Ambroise Lumanikio | OneTechly

1. The Whole Stack on One Page

Every row is a separate piece of software with one job. Most interview questions about a backend are really the question "which layer does that, and why is it a separate layer?"

Layer	In Brief!	Its one job	Where to look
Language and runtime	Java 17 (JDK)	Compiles and runs the code	`pom.xml` (java.version)
Build tool	Maven, through the Maven Wrapper	Downloads libraries, compiles, runs tests, builds the jar	`pom.xml, mvnw.cmd`
Container	Spring Framework 6.1	Creates the application's objects and connects them	every `@Service`, `@Component`
Start-up and defaults	Spring Boot 3.3.5	Configures everything from what is on the classpath; embeds the web server	`BriefApplication.java, application.properties`
Web server	Tomcat 10.1 (embedded)	Accepts HTTP connections, one thread per request	nothing to write
Web framework	Spring MVC	Turns an HTTP request into a call to one Java method	`controller/`
JSON	Jackson	Converts between JSON text and Java objects	`@JsonProperty` in `entity/, dto/`
Input rules	Jakarta Bean Validation (Hibernate Validator)	Checks request bodies before your code runs	`dto/Credentials.java`
Security	Spring Security 6.3	A chain of filters in front of every controller	`config/SecurityConfig.java, security/`
Login tokens	JWT, signed with HMAC-SHA256 (JDK)	Proves who is calling without server-side sessions	`security/JwtService.java`
Password storage	PBKDF2-HMAC-SHA256 (JDK)	Stores a slow, salted hash instead of the password	`util/PasswordHasher.java`
Repositories	Spring Data JPA	Writes the data-access classes for you from interfaces	`repository/`
ORM	Hibernate 6.5 (implements JPA)	Maps objects to table rows and generates the SQL	`entity/`
Connection pool	HikariCP	Keeps database connections open and lends them out	nothing to write
Database driver API	JDBC	The standard Java way to send SQL to any database	used directly only in the Javalin edition
Database	H2 by default, MySQL by profile	Stores the rows	`application-mysql.properties`
Tests	JUnit 5, Mockito, Spring Boot Test	Prove the rules hold, with and without the server	`src/test/java`

One request, start to finish

A signed-in member posts a message: `POST /messages`. In order:

1. **Tomcat** accepts the connection and hands the request to a worker thread.

2. **Spring Security's filter chain** runs. `JwtAuthenticationFilter` reads the `Authorization: Bearer` header, checks the token's signature, and records the caller's account id in the `SecurityContext`. The URL rule for `POST /messages` then requires a signed-in caller; without one the answer is 401 and nothing below runs.
3. **Spring MVC's DispatcherServlet** finds the method mapped to this URL: `MessageController.create`.
4. **Jackson** builds a `MessageRequest` record from the JSON body. **Bean Validation** checks it (`@NotBlank`, `@Size(max = 255)`); a broken rule is a 400.
5. **The controller** asks `AuthGuard` whether this caller may post as that account (else 403), then calls the service.
6. **MessageService.create** is `@Transactional`: Spring opens a database transaction around it. It checks that the author exists, saves the message, and asks `MentionService` to record who is mentioned.
7. **Spring Data JPA and Hibernate** turn `save(...)` into an SQL `INSERT`, sent over a **JDBC** connection borrowed from **HikariCP**.
8. The method returns; the transaction commits. Jackson turns the returned `Message` into JSON. Status 200.

> **Say it in an interview**
>
> "A request passes through Tomcat, the Spring Security filter chain, then Spring MVC routes it to a controller. The controller validates and delegates to a service, the service holds the business rules and the transaction, and a Spring Data repository backed by Hibernate talks to the database."

2. Spring Framework: IoC, Beans, Dependency Injection

Spring Framework is, at its centre, one thing: a **container** that creates your objects and connects them to each other. Everything else in Spring is built on that.

Term	Meaning	In Brief!
Inversion of Control (IoC)	Your classes no longer create the objects they depend on. A container creates them and hands them over. Control over "who builds what" is inverted: it moves from your code to the framework.	No class in the Spring edition calls `new AccountService(...)`
IoC container	The object that does this. In Spring it is the `ApplicationContext`.	Created by `SpringApplication.run(...)`
Bean	An object the container creates and manages.	`AccountService`, `JwtService`, `AuthGuard`, every controller, every repository
Dependency Injection (DI)	How the container hands a bean what it needs: it passes the other beans in, normally through the constructor. DI is the technique; IoC is the principle.	The constructor of `MessageService` asks for two repositories and a `MentionService`
Component scanning	At start-up Spring looks through your packages for annotated classes and registers each as a bean.	Starts at the package of `BriefApplication`
Scope	How many instances exist. The default is **singleton**: exactly one, shared by everyone.	One `LoginRateLimiter` serves all requests

© 2026 Ambroise Lumanikio | OneTechly

Instantiating versus injecting

You wrote both. The Javalin edition builds its own collaborators; the Spring edition declares what it needs.

Javalin edition: you instantiate (Controller/SocialMediaController.java, shortened)

```
public SocialMediaController(boolean strict) {
    this(new AccountService(new AccountDAO()),
         new MessageService(new MessageDAO()),
         new SessionService(),
         new MentionService(new AccountDAO(),
                            new MessageDAO()),
         strict);
}

// The controller decides which classes to use,
// creates them, and creates what THEY need.
```

Spring edition: the container injects (service/MessageService.java)

```
@Service
public class MessageService {
  private final MessageRepository messages;
  private final AccountRepository accounts;
  private final MentionService mentionService;

  public MessageService(
        MessageRepository messages,
        AccountRepository accounts,
        MentionService mentionService) {
    this.messages = messages;
    this.accounts = accounts;
    this.mentionService = mentionService;
  }
```

Question	Instantiate (`new`)	Inject (DI)
Who calls `new`?	Your class	The container, once, at start-up
Who decides which implementation is used?	The class that needs it. It is welded to that choice.	Configuration. The class only names a type.
How many instances?	As many as there are `new` calls. Above, two `AccountDAO` objects exist by accident.	One, shared (singleton)
In a unit test	Hard to swap a part unless a second constructor was added for it	Call the constructor yourself and pass a mock: `new MessageService(mock, mock, ...)`
Cost	None: you can read every line that runs	Less visible: wiring happens at start-up, and a missing bean is a start-up error, not a compile error

How a class becomes a bean

Annotation	Use it on	Brief! example
`@Component`	Any class the container should manage. The general form.	`AuthGuard, AuthMode, PasswordPolicy, LoginRateLimiter`
`@Service`	A class holding business rules. Same effect as `@Component`; the name documents the role.	`AccountService, MessageService, MentionService, JwtService`
`@RestController`	A class whose methods answer HTTP requests with JSON.	`AccountController, MessageController`
`@RestControllerAdvice`	A class that handles exceptions for all controllers.	`ApiExceptionHandler`
(interface extends `JpaRepository`)	No annotation needed. Spring Data generates the class and registers it.	`AccountRepository, MessageRepository, MentionRepository`
`@Configuration` + `@Bean`	When you must build the object yourself, in a method. The method's return value becomes the bean.	`SecurityConfig.securityFilterChain(...)`

© 2026 Ambroise Lumanikio | OneTechly

Three things worth knowing beyond the definition

- **Constructor injection is the preferred form.** Fields can be `final`, an object can never exist half-built, and a test can create it with plain `new`. With one constructor, no annotation is needed. With two, mark the one Spring should use with `@Autowired` (`JwtService`, `LoginRateLimiter`).
- **Singleton means shared between threads.** One `MessageService` handles every request at the same time, so a service must not keep per-request data in a field. Where shared data is the point, it must be thread-safe: `LoginRateLimiter` uses `ConcurrentHashMap.compute`.
- **Configuration is injected too.** `@Value("${brief.auth.mode:strict}")` puts a property into a constructor parameter; the part after the colon is the default (`AuthMode`).

> **Say it in an interview**
>
> "Inversion of Control means my classes do not create their dependencies; the Spring container does, and passes them in through the constructor. That is dependency injection. A bean is any object the container manages. The benefit is loose coupling: I can hand a service a mock repository in a test without changing the service."

3. Spring Boot

Spring Framework gives you the container and the libraries. **Spring Boot** is the layer that sets them up so that an application starts with almost no configuration. It is not a different framework; it is Spring, pre-assembled.

Spring Boot adds	What that means	In Brief!
Starters	One dependency that brings a whole working set of libraries in versions known to work together.	`spring-boot-starter-web, -data-jpa, -validation, -security, -test`
Auto-configuration	At start-up Boot looks at the classpath and creates the beans you would otherwise write by hand. It sees a JDBC driver and JPA, so it creates a connection pool and Hibernate's `EntityManagerFactory`.	You wrote no database set-up code
It backs off	Each auto-configured bean is conditional: "only if the developer has not defined one." Define your own and Boot's default disappears.	Your `SecurityFilterChain` replaces Boot's lock-everything default
Embedded server	Tomcat is a library inside your jar, started by your `main` method. No server to install.	Log line: "Tomcat started on port 8080"
Externalized configuration	`application.properties`, overridable by environment variables and command-line arguments, in that rising order of priority.	`server.port=${PORT:8080}`
Profiles	Named sets of overrides, switched on at start-up.	`--spring.profiles.active=mysql` loads `application-mysql.properties`
Executable jar	The Spring Boot Maven plugin packs your classes and every library into one file.	`java -jar target\brief-spring.jar`
Starter parent	A parent `pom.xml` that fixes library versions, so your dependencies need no version numbers.	`<parent>spring-boot-starter-parent 3.3.5`

The one annotation that starts it

```
@SpringBootApplication          // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class BriefApplication {
    public static void main(String[] args) {
        SpringApplication.run(BriefApplication.class, args);   // builds the container, starts Tomcat
    }
}
```

Compare with `Main.java` in the Javalin edition, where creating the schema, reading the settings, building the controller and starting the server are four visible steps. Boot does the same four things; you just do not see them.

> **Say it in an interview**
>
> "Spring Boot is Spring with auto-configuration, starter dependencies and an embedded server. It reads the classpath and my properties file and configures the application for me, and it steps aside wherever I define a bean of my own. The result is one runnable jar."

4. The Web Layer: Spring MVC, JSON, Validation

Spring MVC has one servlet, the `DispatcherServlet`, that receives every request and dispatches it to the controller method whose mapping matches. Your methods take plain Java parameters and return plain Java objects; the annotations say where each value comes from.

Annotation	Meaning	Brief! example
`@GetMapping, @PostMapping, @PatchMapping, @DeleteMapping`	This method answers that HTTP method and path.	`@PatchMapping("/messages/{messageId}")`
`@RequestBody`	Build this parameter from the JSON body (Jackson does it).	`@RequestBody Credentials credentials`
`@PathVariable`	Take it from the `{...}` part of the URL.	`/accounts/{accountId}`
`@RequestParam`	Take it from the query string.	`?limit=10&before=42`
`@RequestHeader`	Take it from a request header.	`Authorization` in `logout`
`@Valid`	Check the constraints on this object first. If one fails, the method is never called.	`register(@Valid @RequestBody ...)`
`ResponseEntity<T>`	Return type when you need to set the status or a header, not only the body.	login returns the token in `X-Auth-Token`

DTOs, records and validation

- A **DTO** (data transfer object) is the shape of a request or response, kept separate from the entity that maps to the table. A client therefore cannot set fields it should not (`message_id`), and the table can change without changing the API. Brief! uses Java **records** for them: `Credentials, MessageRequest, PublicAccount`.
- **Bean Validation** puts the rule on the data: `@NotBlank @Size(max = 255) String messageText`. The Javalin edition has the same rule as an `if` statement in the service that someone must remember to call.
- Rules that depend on configuration or on the database cannot be annotations. They live in beans: `PasswordPolicy` (strict or compat mode), `MessageService` (does the author exist?).

One place for errors

Services signal a broken rule by **throwing**. `ApiExceptionHandler` (`@RestControllerAdvice`) catches each exception type for every controller and answers with a status code and an empty body.

© 2026 Ambroise Lumanikio | OneTechly

Status	Meaning	Raised in Brief! when
200	It worked	Also for "nothing there" on GET and DELETE of a message: the original contract
400 Bad Request	The request is wrong	Invalid JSON, a broken validation rule, an unknown author, a weak password, a taken username
401 Unauthorized	I do not know who you are	Wrong login; missing, altered, expired or signed-out token
403 Forbidden	I know who you are, and you may not	Posting as someone else; editing or deleting someone else's post
404 Not Found	No such thing	Unknown account
409 Conflict	Collided with another change	Two requests writing the same row at the same instant
429 Too Many Requests	Slow down	Too many login attempts; `Retry-After` says how long

Say it in an interview

"Controllers stay thin: they map HTTP to a method call. Validation is declared on record DTOs and runs before the controller. Services throw typed exceptions, and one @RestControllerAdvice class maps them to status codes, so error handling is in one place instead of every handler."

5. Persistence: JDBC, JPA, Hibernate, Spring Data JPA

Four names that are often confused. They are four layers, each built on the one below. You have written against the bottom one (Javalin edition) and the top one (Spring edition).

Name	What it is	What you write with it	Brief!
JDBC	The standard Java API for sending SQL to a database. Part of the JDK. Each database supplies a driver.	SQL strings, `PreparedStatement`, `ResultSet`, copying columns into objects by hand	Javalin edition: `DAO/AccountDAO.java`
JPA (Jakarta Persistence)	A **specification**: a set of annotations and interfaces for mapping objects to tables. It contains no working code.	`@Entity, @Id, @Column, @Table` on a class	`entity/Account.java, Message.java, Mention.java`
Hibernate	The **implementation** of JPA: an object-relational mapper (ORM). It reads the annotations and generates the SQL, through JDBC.	Nothing directly. It works underneath.	Log line: "Hibernate ORM core version 6.5.3"
Spring Data JPA	A layer on top of JPA that writes the repository classes for you from interfaces.	An interface that extends `JpaRepository`; method names that become queries	`repository/`

The same query at both ends of that table:

JDBC, by hand (Javalin edition, shortened)

```
String sql = "SELECT * FROM message"
           + " WHERE posted_by = ?"
           + " ORDER BY message_id";
try (Connection c = pool.getConnection();
     PreparedStatement ps =
             c.prepareStatement(sql)) {
  ps.setInt(1, accountId);
  try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
      list.add(mapRow(rs));   // column -> field
    }
  }
}
```

Spring Data JPA (Spring edition)

```
public interface MessageRepository
    extends JpaRepository<Message, Integer> {

  List<Message>
  findByPostedByOrderByMessageIdAsc(
      Integer postedBy);
}

// No implementation class exists in the
// project. Spring Data generates one at
// start-up, and Hibernate writes the SQL.
```

Three ways to get a query

Way	How	Brief! example
Inherited	`JpaRepository` already has `save`, `findById`, `existsById`, `findAllById`, `delete`, `count`.	`accounts.existsById(id)`
Derived from the method name	Spring Data parses the name: `find | By PostedBy | OrderBy MessageId Asc`. The words must be field names of the entity. A typo is found at start-up, not by the compiler.	`findByMessageIdLessThanOrderByMessageId Desc(id, pageable)`
Written out with `@Query`	JPQL: looks like SQL but names the Java class and fields, not the table and columns. Hibernate translates it for whichever database is in use.	`select a from Account a where locate(lower(a.username), lower(:text)) > 0`

Words you will be asked about

- **Entity.** A class whose objects are rows. `@Id` marks the primary key; `@GeneratedValue(strategy = IDENTITY)` lets the database assign it.
- **ORM.** Object-relational mapping: you describe the mapping once with annotations and the mapper writes the SQL. You give up some visibility and control in exchange for far less code.
- `save`**.** Inserts an object that has no id yet; updates one that has.
- `Optional<T>`**.** `findById` returns it instead of `null`, so the caller is forced to handle "no such row".
- **Paging.** A `Pageable` parameter adds the row limit. Brief! uses cursor paging: "the next 10 below id 42", and fetches one extra row to learn whether more exist.
- **SQL injection.** Prevented at both ends for the same reason: values travel as bound parameters (`?` or `:name`), never glued into the SQL text.
- **N+1 queries.** Fetching a list, then one more query per item. `MentionService` avoids it with `findAllById`: one query for all the messages.
- `spring.jpa.hibernate.ddl-auto=update`**.** Hibernate creates missing tables and columns from the entities. Convenient in development. In production you set it to `validate` and change the schema with versioned migration scripts (Flyway), because `update` never drops or renames anything and leaves no record of what changed.
- **H2 and MySQL.** H2 is a database written in Java that runs inside the application: nothing to install, ideal for development and tests. Switching to MySQL is one profile, because Hibernate writes the SQL in the right dialect. *The MySQL profile in Brief! has not been run yet; say so if asked.*

© 2026 Ambroise Lumanikio | OneTechly

> **Say it in an interview**
>
> "JDBC is the low-level API for sending SQL. JPA is the specification for mapping objects to tables, Hibernate is the implementation that generates the SQL, and Spring Data JPA sits on top and generates repositories from interfaces. I wrote the same data layer twice, once in plain JDBC and once with Spring Data, so I know what the framework is doing for me."

6. Transactions, Indexes, Constraints

Version 2.2 moved mentions into their own table. That one change needs all three ideas on this page.

Transactions

A **transaction** is a group of database changes that succeed or fail as one. Posting a message now writes two things: the message row, and one `mention` row for each member it names. Without a transaction, a failure between the two would leave a message whose mentions were never recorded.

```
@Transactional
public Message create(MessageRequest request) {
    ...
    Message saved = messages.save(new Message(request.postedBy(), request.messageText(), time));
    mentionService.index(saved);     // joins the same transaction
    return saved;                    // normal return: COMMIT.  RuntimeException: ROLLBACK of both.
}
```

- **How it works.** Spring does not give other classes your `MessageService` directly. It gives them a **proxy**: a generated object that begins a transaction, calls your method, then commits or rolls back.
- **The consequence.** The annotation only takes effect when the method is called from *outside* the class, through the proxy. A method calling another method of its own class bypasses it.
- **Rollback rule.** By default a transaction rolls back on a `RuntimeException` and commits on a checked exception.
- **Propagation.** `MentionService.index` is also `@Transactional`. Called inside a running transaction it joins that one (the default, `REQUIRED`).
- **ACID**, if asked: Atomic (all or nothing), Consistent (rules hold before and after), Isolated (concurrent transactions do not see each other's half-done work), Durable (once committed, it survives a crash).

Indexes and unique constraints

Thing	What it is	In `entity/Mention.java`
Index	A sorted copy of chosen columns that the database keeps beside the table, like the index of a book. A search on those columns no longer reads every row. The price: each insert and delete must update the index too.	`(account_id, message_id)`: "all mentions of account 7, in order" is a direct look-up
Composite index	An index on several columns. Order matters: it helps a search by the first column, or the first and second, but not by the second alone.	Helps "by account"; would not help "by message" alone
Unique constraint	A rule the database itself enforces: no two rows with the same values in these columns. It holds even if the code has a bug or two requests race.	`(message_id, account_id)`: a post mentions a member at most once
Primary key	The column that identifies a row. Always unique, always indexed.	`mention_id`

The scaling decision, in one table

	Search when asked (up to 2.1)	Store when written (2.2)
"Who mentioned me?"	Reads every message and searches its text	Reads only that member's rows, through the index
Asked how often	Every 15 seconds, by every signed-in browser	(same)
Cost of a new post	Nothing extra	One query to find the mentioned members, a few inserts
Grows with	Members online x messages ever written	How often that one member is mentioned

Say it in an interview

"The mentions feature originally scanned every message on every poll. I moved that work to write time: when a post is saved, the members it mentions go into a mention table with a composite index, inside the same transaction as the post. Reads became an index look-up. It is the standard trade: a little more work per write for much cheaper, far more frequent reads."

7. Security: Passwords, Spring Security, JWT, Rate Limiting

Two words first. **Authentication** is finding out who is calling; failing it is a 401. **Authorization** is deciding whether that caller may do this; failing it is a 403.

Storing passwords: PBKDF2 with HMAC-SHA256

A password must never be stored, and must never be stored merely encrypted either: encryption can be reversed by whoever holds the key. It is **hashed**: put through a one-way function. At login the same function is run on what was typed and the two results are compared.

Ingredient	Why	In `util/PasswordHasher.java`
One-way hash	The stored value cannot be turned back into the password.	HMAC-SHA256 is the building block
Salt	Random bytes, different for every account, mixed in before hashing. Two members with the same password get different hashes, and a table of pre-computed hashes is useless.	16 random bytes from `SecureRandom`
Deliberately slow	A plain SHA-256 can be tried billions of times a second on stolen data. A key-derivation function repeats the hash so each guess costs real time.	120,000 iterations (the five hashing tests take about a second in your test log)
Constant-time comparison	Compare all bytes always, so the response time does not reveal how many were right.	`MessageDigest.isEqual`
Self-describing format	Store the method and its parameters with the hash, so they can be raised later without breaking old accounts.	`pbkdf2$120000$<salt>$<hash>`

- **PBKDF2** stands for Password-Based Key Derivation Function 2, a published standard (RFC 8018). "With HMAC-SHA256" names the hash it repeats.
- **"From the JDK - no extra dependency"** means the algorithm ships with Java itself: `SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")`. Nothing is added to `pom.xml`, so there is no third-party library to keep patched.
- **Alternatives you should be able to name:** bcrypt, scrypt and Argon2. Spring Security offers them behind its `PasswordEncoder` interface, and a Spring team would normally use that. The idea is identical: salted, slow,

one-way.

- **Migrate on login.** An old plain-text row is accepted once, then replaced by a hash (`AccountService.login`). That is how a real system upgrades stored passwords without asking everyone to reset.

Spring Security

Spring Security is a **chain of servlet filters** that every request passes through before it reaches a controller. You describe the chain once, as a bean, in `config/SecurityConfig.java`.

Setting in Brief!	Why
`csrf.disable()`	CSRF attacks abuse cookies that the browser attaches by itself. Brief! has no cookies: the token travels in a header that another site cannot add.
`sessionCreationPolicy(STATELESS)`	No server-side session. Every request proves itself with its token, so any server instance can answer it.
`logout.disable()`	Spring Security's own `/logout` handler would answer before the application's does.
`authenticationEntryPoint(401)`	An API should answer "not signed in" with a bare 401, not a redirect to a login page.
URL rules: POST, PATCH, DELETE on messages need a signed-in caller	"Is anyone signed in?" can be decided from the URL alone, before the controller runs.
`addFilterBefore(new JwtAuthenticationFilter(...))`	Our filter reads the token and fills the `SecurityContext` (who is calling, for this request).
`AuthGuard` (your own class)	"Is it *your* post?" needs the database, so it cannot be a URL rule. It reads the caller from the `SecurityContext`.

JWT (JSON Web Token)

```
header . payload . signature          each part Base64URL-encoded, joined by dots

header     {"alg":"HS256","typ":"JWT"}                                  how it is signed
payload    {"sub":"7","jti":"...","iat":1791250000,"exp":1791293200}    who, token id, issued, expires
signature  HMAC-SHA256(header + "." + payload, secret key)              key known only to the server
```

- The payload is **readable by anyone**; it is encoded, not encrypted. Nothing secret goes in it.
- The signature gives **integrity**: change one character of the payload and the signature no longer matches. Only the holder of the key can produce a valid one.
- The server checks a token with the key alone. It looks nothing up. That is what **stateless** means, and why several servers can share the load if they share the key.
- **Verify first, read second.** `JwtService` checks the signature and only then parses the payload.
- **The catch:** a signed token stays valid until it expires. Sign-out therefore keeps a small list of revoked token ids.
- **The key is configuration**, set in the environment (`BRIEF_JWT_SECRET`), never written in a file that is committed.
- Brief! signs with the JDK so every step is visible. A team would normally use a library (jjwt, or Spring's OAuth2 resource server). Say that before you are asked.

Rate limiting

`LoginRateLimiter` counts login attempts per (address, username). After five without a success it answers **429** with a `Retry-After` header for 15 minutes. The attempt is counted *before* the password is checked, inside `ConcurrentHashMap.compute`, so fifty guesses arriving at the same instant cannot all slip past the check. Its honest limit: the counts are in one server's memory. More than one server needs a shared store such as Redis.

> **Say it in an interview**
>
> "Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes with 120,000 iterations, using the JDK's own implementation. Login returns a JWT signed with HMAC-SHA256. A Spring Security filter validates it on every request and puts the caller in the SecurityContext. URL rules decide who must be signed in, and a guard checks ownership. Logins are rate limited per address and username."

8. Testing: JUnit 5, Mockito, Spring Boot Test

Kind	What runs	Speed	In Brief!
Unit test	One class, created with `new`. Its collaborators are replaced by mocks. No Spring, no database, no network.	Milliseconds	86 tests in `unit/`
End-to-end (integration) test	The whole application: every bean, a real Tomcat on a random port, a real in-memory database. The test sends real HTTP requests.	Seconds to start, then fast	63 tests in `integration/`

Tool	What it gives you	Seen in
JUnit 5	`@Test` marks a test; `@BeforeEach` / `@AfterEach` run around each one; `assertEquals, assertTrue, assertThrows` state what must be true.	every test class
Mockito	`mock(X.class)` makes a stand-in. `when(...).thenReturn(...)` tells it what to answer. `verify(...)` checks a call happened. `ArgumentCaptor` keeps what was passed. `thenAnswer` runs code.	`AccountServiceTest`, `MessageServiceTest`, `MentionServiceTest`
Spring Boot Test	`@SpringBootTest(webEnvironment = RANDOM_PORT)` starts the real application for the test class. `properties = {...}` overrides settings for that class only.	`ContractTest` (compat mode), `StrictAuthTest` (strict mode)
Spring's mock servlet objects	`MockHttpServletRequest` and friends let a filter be tested as a plain method call.	`JwtAuthenticationFilterTest`

Techniques in these tests that are worth naming

- **An injectable clock.** `JwtService` and `LoginRateLimiter` never read the system time directly; they are given a clock. The test passes a counter, so "15 minutes later" is one line, not a 15-minute wait.
- **Tests that play the attacker.** Change a token's payload and keep its signature; send a token signed with another key; send 40 login attempts from 40 threads at once. Security code is tested by trying to break it.
- **A fake.** In `MentionServiceTest` the mock repository is given a memory (a list). The test then checks what ended up stored, not which methods were called, so it survives a reorganisation of the code.
- **An isolated database.** Tests run against an in-memory H2 and reset it before each test. Your real data is never touched.
- **A test that cannot fail proves nothing.** Before each release the code was broken on purpose in several ways, to confirm that a test goes red each time.

> **Say it in an interview**
>
> "I have fast unit tests for the rules, with Mockito standing in for the repositories, and end-to-end tests that start the whole Spring Boot application on a random port against an in-memory database and call it over real HTTP. The current count is 149: 86 unit and 63 end-to-end."

© 2026 Ambroise Lumanikio | OneTechly

9. Maven and Java 17

Maven is the build tool. Everything it does is driven by `pom.xml`.

In `pom.xml`	Meaning
`<parent>`	Inherit version numbers and plugin settings from `spring-boot-starter-parent`.
`<dependency>`	A library to download from Maven Central, together with the libraries it needs in turn (transitive dependencies).
`<scope>`	`runtime`: needed to run, not to compile (the H2 and MySQL drivers). `test`: only for tests (JUnit, Mockito).
`<plugin>`	Something that takes part in the build. `spring-boot-maven-plugin` builds the runnable jar.

Command	What happens
`.\mvnw.cmd test`	Compile, then run every test. The lifecycle runs in order: validate, compile, test.
`.\mvnw.cmd package`	All of the above, then build `target\brief-spring.jar`.
`java -jar target\brief-spring.jar`	Start the application. Maven is no longer involved.
`mvnw` (the Maven Wrapper)	A script kept in the project that downloads the right Maven version. Nobody has to install Maven.

Java language features the code relies on

Feature	Since	Example in Brief!
Records	Java 16	`public record Credentials(String username, String password)`: an immutable data class in one line
Pattern matching for `instanceof`	Java 16	`if (auth.getPrincipal() instanceof Integer accountId)` in `AuthGuard`
`Optional<T>`	Java 8	`accounts.findByUsername(name).orElseThrow(UnauthorizedException::new)`
Lambdas and method references	Java 8	`csrf -> csrf.disable()`, `Message::getMessageId`
Generics	Java 5	`JpaRepository<Message, Integer>`, `List<Message>`
`java.util.concurrent`	Java 5 / 8	`ConcurrentHashMap.compute` in `LoginRateLimiter`
try-with-resources	Java 7	Javalin edition DAOs: connections close themselves

© 2026 Ambroise Lumanikio | OneTechly

10. Interview Answers in Two Sentences

Each answer is true of your own code. Add the file name if they want detail.

If they ask	You can say
What is Spring Boot?	Spring with auto-configuration, starter dependencies and an embedded server. It configures the application from the classpath and a properties file, and steps aside where I define my own beans.
What is dependency injection?	A class declares what it needs in its constructor and the Spring container supplies it. The class does not build its own dependencies, so I can swap in a mock for a test.
What is a bean?	An object the Spring container creates and manages. By default there is one instance, shared by all request threads, so beans must not hold per-request state.
Why constructor injection?	The fields can be final, the object is never half-built, and a test can create it with plain `new`.
JPA, Hibernate, Spring Data JPA?	JPA is the specification, Hibernate implements it and generates the SQL, Spring Data JPA generates repositories on top. Underneath all of them is JDBC, which I also used directly in the first edition.
What does `@Transactional` do?	It wraps the method in a database transaction through a proxy: commit on return, rollback on a runtime exception. I use it so a post and its mention rows are saved together or not at all.
How are passwords stored?	As salted PBKDF2-HMAC-SHA256 hashes, 120,000 iterations, compared in constant time. Never the password, and the format records its own parameters so they can be raised later.
How does login work?	Login returns a JWT signed with HMAC-SHA256. A Spring Security filter verifies it on each request, so the server keeps no session. Sign-out uses a revocation list, because a signed token cannot be un-issued.
401 or 403?	401: I do not know who you are. 403: I know who you are, and you may not do this.
How do you prevent SQL injection?	Values are always bound parameters, never concatenated into SQL. That holds for my PreparedStatements, for Spring Data's derived queries, and for the one JPQL query I wrote.
What does idempotent mean?	Doing it twice has the same effect as doing it once. `DELETE /messages/{id}` answers 200 the second time too, with an empty body.
How does paging work?	Cursor paging: the client asks for the next ten below the oldest id it has. New posts cannot shift the pages, and I fetch one extra row to know whether more exist.
How would you scale it?	It is already stateless, so more instances can share the load once the two in-memory pieces, the rate-limit counters and the revoked-token list, move to Redis. The mention look-up is indexed. Next would be a managed database with versioned migrations.
What did you learn from building it twice?	What the framework actually does. Every annotation in the Spring edition replaces code I had already written by hand in the Javalin edition, so none of it is magic to me.
What is not done yet?	The MySQL profile is untested, schema changes are not versioned (no Flyway yet), there is no email on accounts so no password reset, and it has not been deployed. I know the order I would do them in.

Stay exact

These claims are accurate for Brief! 2.2.0. Two are easy to overstate, so keep them exact: the Spring edition's 2.2.0 tests have not been run on your machine yet at the time of writing (2.1.0 passed 133 of 133), and the JWT and rate-limit code are your own teaching implementations, not a library.

© 2026 Ambroise Lumanikio | OneTechly